



Analysis of Hardware and Software (Open-Source and Proprietary) for Terrain Mapping and Object Detection in Unmanned Aerial Vehicles – Research Report

Objective

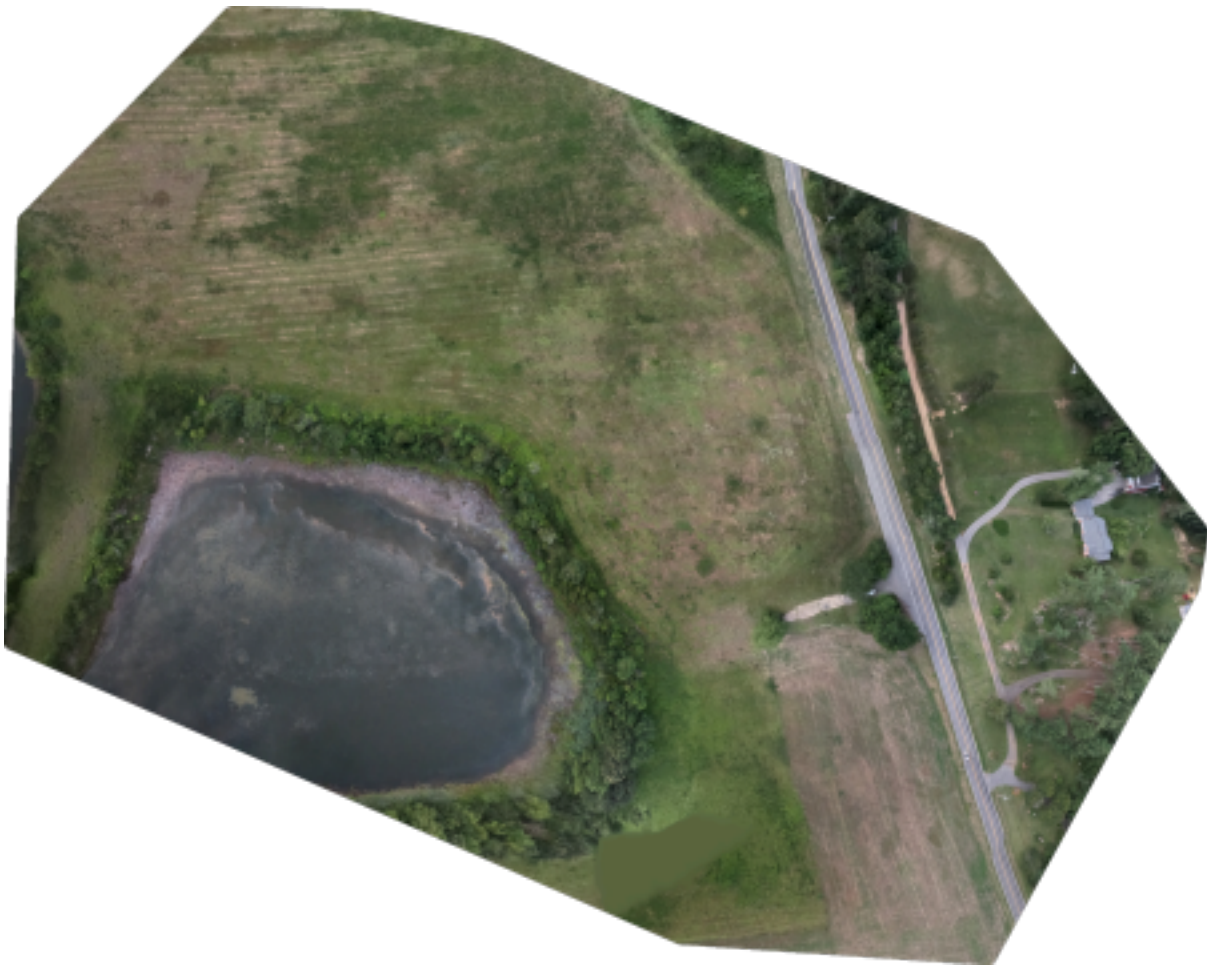
The aim of the study was to analyze available open-source libraries, implement proprietary solutions, and select appropriate tools enabling terrain mapping and object detection using unmanned aerial vehicles (UAVs). The work included the development of custom software and a comparison of its performance with existing solutions. Additionally, appropriate hardware was selected, including cameras and computing units (onboard microcomputers and ground stations), for the execution of specific tasks. Test missions were designed based on the requirements of the SUAS 2025 (United States) and TEKNOFEST 2025 (Turkey) competitions. The research was carried out using two types of aerial vehicles—a fixed-wing aircraft and a multirotor drone—in order to assess the effectiveness of the solutions in different operational scenarios.

Terrain Mapping

Research Process

One of the competition tasks at SUAS 2025 was to map a designated area using a UAV. The team participated with a fixed-wing UAV. This required a camera capable of recording high-quality video, resistant to distortions caused by high flight speeds (up to 30 m/s) or maneuvers typical for fixed-wing aircraft. For this reason, a GoPro 11 camera was chosen, high image quality with adequate stabilization while staying within the allocated grant budget. The first stage of the work was the creation of a prototype of custom image-processing software utilizing OpenCV library in Python, which is suitable for planning and prototyping. Initial tests showed that the performance of this solution was insufficient for the required image quality and the number of frames generated by the camera. Therefore, the final implementation was developed in C++, which allowed for shorter computation times and greater control over parameters. The software was optimized iteratively and tested under various terrain conditions (grassy areas, asphalt surfaces, regions with few feature points).

An example map generated during field tests:



Hardware Limitations

Hardware limitations proved to be a major challenge. Due to the small size of the fixed-wing fuselage and its high speed and agility oriented design, installing a powerful onboard microcomputer was practically impossible. Smaller embedded systems did not provide sufficient computing power, and their operation caused additional issues: excessive heating, lack of space for auxiliary components (e.g., GPU/AI modules), and potential risks of damaging other electronics. The alternative was to perform computations on the ground station during a short, ten-minute intermediate landing. This solution increased computational capacity at the cost of reducing the available processing time from 30 minutes (entire mission duration) to 10 minutes. Another issue was limited ability to transmit video in real time. The radio environment during the competition (a large number of active signals) interfered with stable connection to the ground station, which ruled out in-flight computations.

Software Selection

Given these limitations, a decision was made to use the OpenDroneMap (ODM) tool. This is a proven open-source solution developed by a large community of experts, featuring advanced and optimized algorithms, implementation of which using OpenCV would have



Fundusze Europejskie
dla Rozwoju Społecznego



Rzeczpospolita
Polska

Dofinansowane przez
Unię Europejską

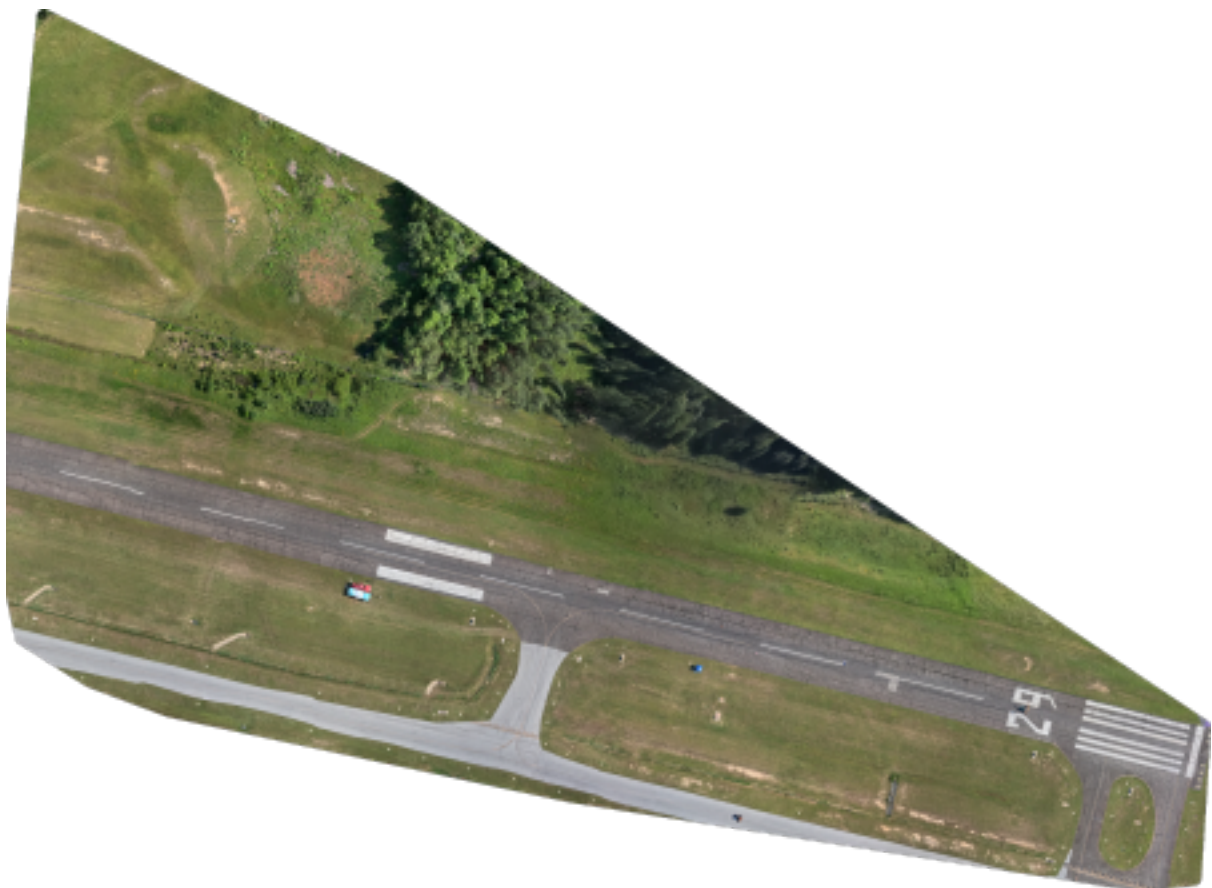


been impractical under the circumstances. Another advantage of ODM was its easy configuration and parameter recording, which allowed for quick adaptation of the mapping process to the specifics of each mission. Computations were carried out on a laptop that was part of the ground station during intermediate landings, providing for an optimal solution under said hardware limitations.

Results

Ultimately, the generated maps were deemed high quality, and the processing time using ODM remained within acceptable limits (under 10 minutes). During the competition run, the team had to evacuate, which made it impossible to redo the calculations during the planned flight time, resulting in no points being awarded for this task. Nonetheless, the map was successfully generated after the mission, confirming the effectiveness of the developed solution.

Final map generated during the SUAS 2025 competition. The area corresponds to the designated waypoints:



Object Detection

Research Process



During TEKNOFEST 2025, the team fielded two squads, each using a different UAV platform—a fixed-wing aircraft and a multirotor. The mission objective was to detect objects in the form of colored geometric shapes. Two approaches to system development were considered: using artificial intelligence with the YOLOv8 model, using traditional image-processing methods with the OpenCV library.

YOLO-Based Approach

At the initial stage, documentation and guides on training and optimizing YOLOv8 models were analyzed. Sources included articles and official documentation (e.g., [Medium: Fine-tuning YOLO for Custom Object Detection – A Complete Guide](#), [docs.ultralytics.com, Ultralytics: Hyperparameter Tuning Guide](#)). Public datasets from Roboflow Universe containing geometric shapes were also used: [Shapes \(ws-qybnz\)](#), [Square Circle Triangle Rectangle](#), [Geometric Shapes \(annotationnations-y2-blockb\)](#). However, these required adjustment, limiting the number of classes used to only those necessary for the mission and manual edition of annotation files. Additional custom data was obtained from UAV footage recorded under competition-like conditions. Advantages of this approach include high detection accuracy, real-time operation, and vast availability of pre-trained models and datasets, while its requirements include high computational demands, time-consuming data preparation process, and the need for extensive fine-tuning.

Classical Approach with OpenCV

The second method relied on traditional image-processing techniques available in OpenCV. This approach involved color segmentation in the HSV space, contour detection, polygon approximation, and shape classification based on the number of vertices. Upon detection, the frame was saved and transmitted to a server. Advantages of this method included simplicity, no need for model training, and low hardware requirements—the algorithm ran efficiently on a CPU with no GPU support. Main drawbacks of this approach include time consuming manual parameter calibration, high sensitivity to lighting and background variations, and susceptibility to false detections.

Selected Solution

After testing both platforms, better results were achieved with the traditional approach. The decisive factor was the low computational cost and the ability to run the algorithm onboard the UAV in real time. For this purpose, a Raspberry Pi 5 (8 GB RAM)—the most powerful available model was selected, offering sufficient performance while maintaining compact size and low cost. The system architecture included: video capture → frame extraction → HSV color segmentation → contour detection and convex hull approximation → shape classification (by vertex count) → centroid calculation → centroid-to-waypoint conversion → UAV trajectory adjustment → transmission of object image to the server.

Camera Selection

The camera is a crucial element of the system. Several options were considered, including the GoPro 11, a dedicated FPV camera, and Raspberry Pi-compatible models. During testing, the GoPro 11 was the first unit considered for the detection system. Despite



excellent image quality and stabilization, integration with the Raspberry Pi proved problematic. GoPro lacks a native, straightforward method for real-time video streaming to Raspberry Pi. Alternative options, such as Wi-Fi or HDMI transmission, required additional adapters and modules. Those solutions were discarded with regard to one or more of the following: increased overall system complexity and failure risk, additional delays in video transmission, complicated real-time analysis, insufficient fuselage space connection stability issues, especially under system load, increased power demand and hardware heating.

Furthermore, the high video quality meant the software had to process an excessive number of high resolution frames, exceeding available computational time. Thus, the GoPro 11 proved impractical for an onboard Raspberry Pi system. Ultimately, the Raspberry Pi Camera HD v3 (12 MPx) was chosen. The deciding factors included: compact size and ease of integration with Raspberry Pi, lower cost sufficient image quality for detection tasks.

A wide-angle camera was also considered. The sensor would have covered a larger area, but strong image distortions led to its rejection in favor of the 76° variant. This required an additional flight pass to cover areas missed during the first pass.

Results

The classical system, based on the OpenCV library, a Raspberry Pi 5 computing unit, and a Pi HD v3 camera, proved stable and able to operate in real time on both UAV platforms. Field tests confirmed correct object detection, and correct automatic flight route modification in relation to shape detection. . Unforeseen difficulties arose during the competition. The multirotor drone crashed during its mission which prevented full system validation on that platform. As for the fixed-wing aircraft, a false detection was recorded - the system mistakenly classified a colorful pattern on a competitor's shirt as a target object. Additionally, target objects were not detected due to their surfaces being highly reflective, altering the color perceived by the software. This incident confirmed the vulnerability of classical detection methods to lighting quality, object texture, and unwanted objects in the camera's field of view.